AI WEATHER WISE API - PROJECT DOCUMENTATION

Date:	25 September 2026	**Team ID:**	SWTID-2026-4307
Project Name:	AI Weather Wise API	**Track:**	AI / ML & Gen AI Track

PROJECT DESCRIPTION

AI Weather Wise API is an intelligent, AI-powered weather forecasting and personal recommendation engine. The application integrates real-time meteorological data with Generative AI (LLMs) to deliver contextual, natural-language travel, activity, and attire recommendations based on dynamic weather conditions.

Utilizing OpenWeatherMap API for live telemetry and OpenAI GPT-4o / Groq LLaMA 3.3-70B via LangChain, the platform converts raw temperature, humidity, wind, and precipitation metrics into human-centric actionable insights. Built with FastAPI and Redis caching, the API delivers sub-second response times while ensuring strict security through API key validation and rate limiting.

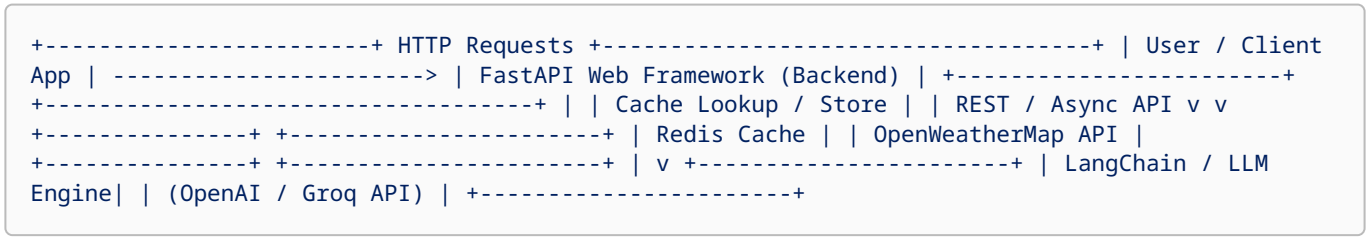
SCENARIOS

Scenario 1: A user queries the weather forecast for London on a rainy weekend. The system fetches live meteorological data and generates tailored indoor activity suggestions, apparel recommendations (e.g., waterproof outerwear), and local transit precautions in conversational prose.

Scenario 2: An event manager requests real-time insights for an outdoor music festival in Miami. The AI Weather Wise API analyzes temperature and UV index spikes, generating safety alerts, hydration reminders, and peak heat avoidance time windows.

Scenario 3: A daily commuter checks morning weather for Tokyo. The API detects sudden evening temperature drops and provides personalized attire adjustments and umbrella reminders straight to their mobile app endpoint.

TECHNICAL ARCHITECTURE



Description: AI Weather Wise API utilizes a high-performance modular microservice architecture. The API frontend handles request routing and JWT/API-key validation. Input queries trigger asynchronous calls to Redis cache; on cache miss, real-time weather metrics are pulled from OpenWeatherMap API. The validated telemetry is then passed to the LangChain LLM engine (OpenAI/Groq) with structured system prompts to generate natural language weather insights and activity suggestions.

PRE-REQUISITES

- **FastAPI Framework Knowledge:** FastAPI Official Documentation
- **OpenWeatherMap API & LLM API Familiarity:** OpenWeather API & OpenAI/Groq Docs
- **Python Programming Proficiency:** Python 3.11+ Documentation
- **Redis In-Memory Database:** Redis Caching Guide
- **Version Control with Git:** Git Documentation
- **Containerization & Deployment:** Docker & Render/AWS Deployment Docs

PROJECT WORKFLOW

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate and securely store OpenWeatherMap and OpenAI/Groq API keys in environment variables.
- **Activity 1.2:** Research, evaluate, and benchmark LLMs for structured prompt execution and latency.
- **Activity 1.3:** Define system architecture detailing API endpoints, Redis caching layer, and LLM orchestration.
- **Activity 1.4:** Set up Python virtual environment and install core framework dependencies.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Implement OpenWeatherMap connector module with error handling and fallback parsing.
- **Activity 2.2:** Build LangChain LLM recommendation engine with structured Pydantic response schemas.

Milestone 3: FastAPI Backend & Router Integration

- **Activity 3.1:** Construct modular FastAPI routes (`/api/v1/weather`, `/api/v1/recommend`).
- **Activity 3.2:** Integrate Redis caching layer to minimize redundant external API requests.

Milestone 4: Testing & Documentation

- **Activity 4.1:** Write comprehensive unit and integration tests using Pytest.
- **Activity 4.2:** Perform load and stress testing using Locust to verify throughput and response times.

Milestone 5: Containerization & Deployment

- **Activity 5.1:** Dockerize application with multi-stage build optimization.
- **Activity 5.2:** Deploy API publicly on Render / AWS EC2 with automatic keep-alive health checks.

DETAILED MILESTONE GUIDE & KEY SETUP SETUP

Milestone 1: API Key & Environment Setup

Before the application can fetch weather data or generate AI recommendations, valid API keys must be created and configured securely.

Step 1 — Obtain API Keys

Visit OpenWeatherMap (console.openweathermap.org) and Groq/OpenAI developers portal to generate free production API keys.

Step 2 — Create .env Configuration File

Create a `.env` file in the root directory of your project (referencing `.env.example`):

```
# .env file configuration
OPENWEATHER_API_KEY=your_openweather_api_key_here
OPENAI_API_KEY=your_openai_or_groq_api_key_here
REDIS_URL=redis://localhost:6379/0
ENVIRONMENT=production
DEBUG=False
```

Step 3 — Load Environment Variables in FastAPI Application

Load environment variables automatically at startup using Pydantic BaseSettings or python-dotenv:

```
from os import os
from dotenv import load_dotenv

load_dotenv()
```

```
OPENWEATHER_API_KEY = os.getenv("OPENWEATHER_API_KEY")
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

if not OPENWEATHER_API_KEY or not OPENAI_API_KEY:
    raise ValueError("Missing critical API keys in environment configuration!")
```

> **Important Security Rule**
>
> Never commit your `.env` file or raw secrets to version control. Always include `.env` in your `.gitignore` file to prevent accidental disclosure.

CONCLUSION

The **AI Weather Wise API** successfully bridges raw meteorological data with Generative AI, offering actionable, context-aware personalized insights. Built on FastAPI, Redis, and LangChain, the application delivers scalable, ultra-fast performance suitable for production deployment across web, mobile, and IoT ecosystems.